

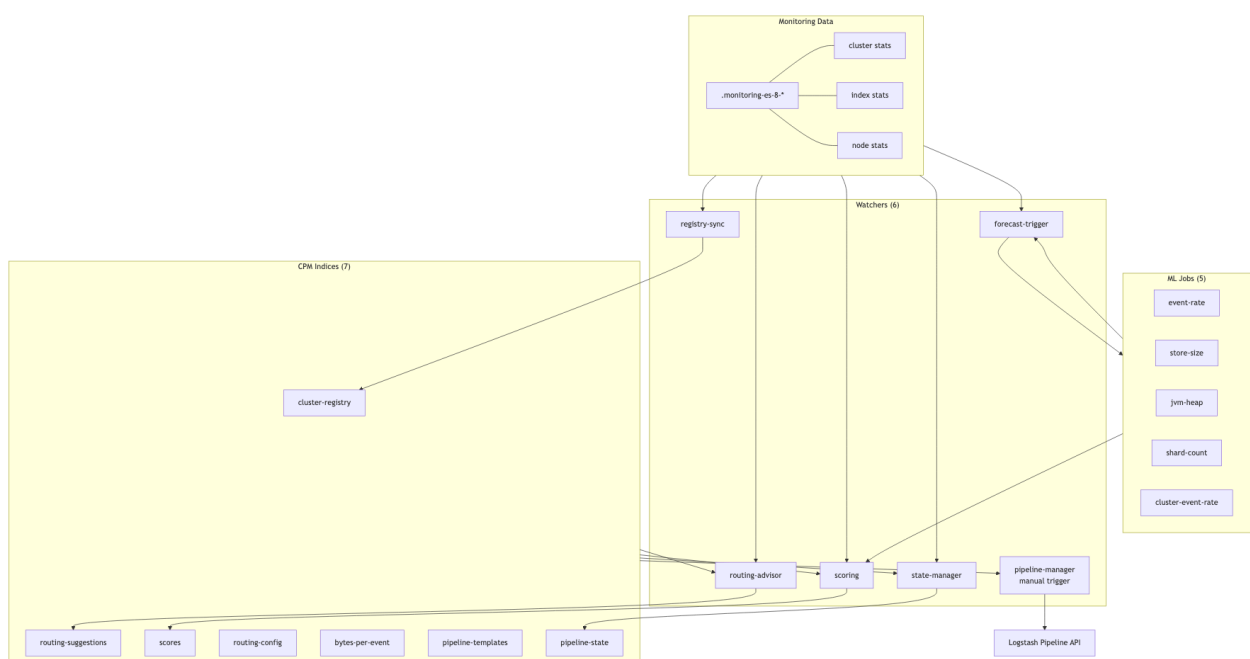
CPM — Cluster Pipeline Manager

CPM automates the management of Logstash ingest pipelines for multi-cluster Elasticsearch deployments. It monitors cluster health, scores clusters based on ML forecasts, recommends dataset-to-cluster routing, computes desired pipeline state, and pushes rendered Logstash pipeline configurations.

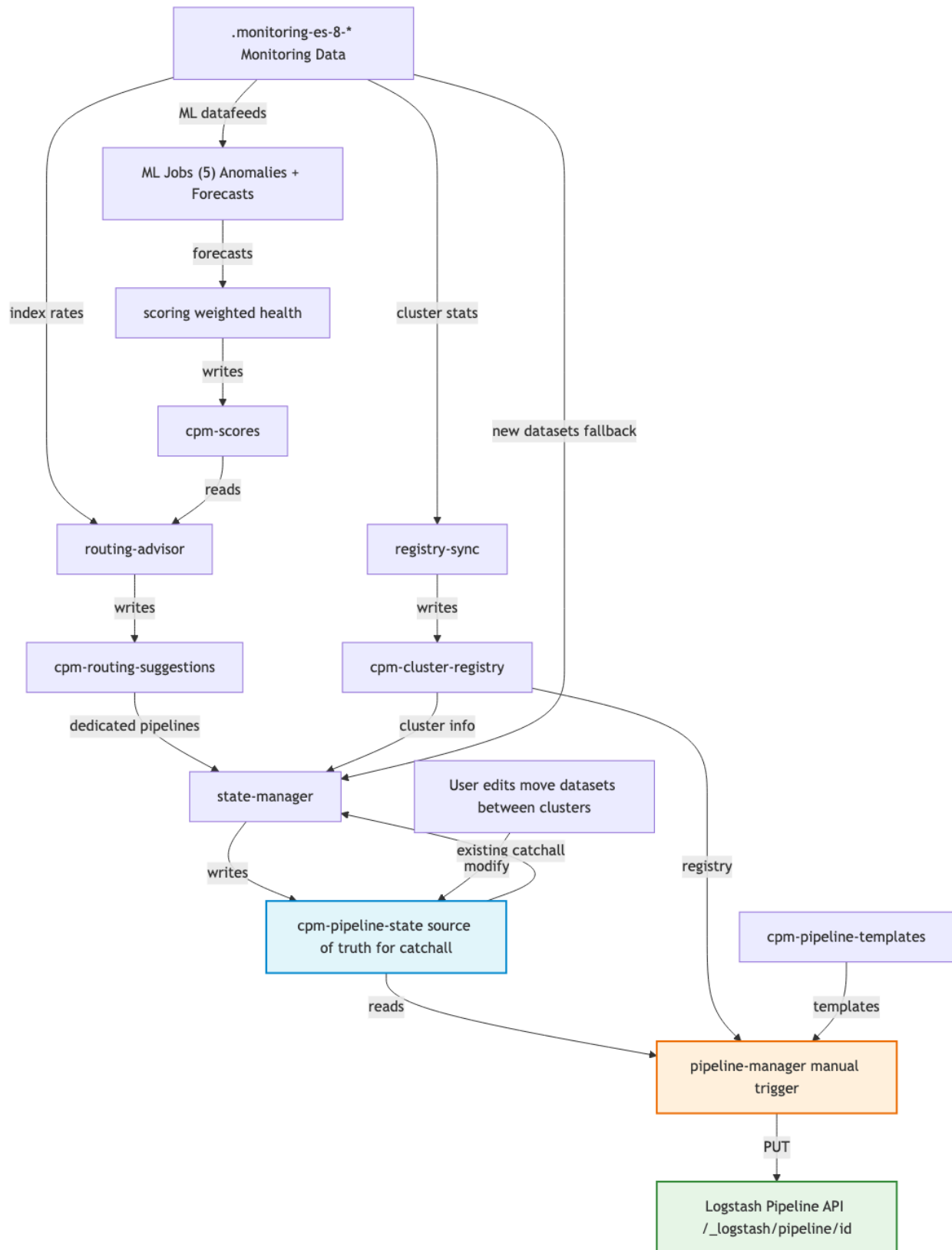
Table of Contents

1. [Architecture Overview](#)
2. [Data Flow](#)
3. [Components](#)
4. [ML Jobs & Datafeeds](#)
5. [Watchers](#)
6. [Indices](#)
7. [Execution Schedule](#)
8. [Pipeline State Model](#)
9. [Template System](#)
10. [Configuration & Deployment](#)
11. [Operational Guide](#)
12. [Painless Development Notes](#)

Architecture Overview



Data Flow



Components

ML Jobs & Datafeeds

Five anomaly detection jobs consume Elasticsearch monitoring data and produce forecasts.

Job	Dataset	Metric	Bucket Span	Model Memory
cpm-event-rate	elasticsearch.index	index_total per backing index	15m	64mb
cpm-store-size	elasticsearch.cluster.stats	store.size.bytes per cluster	15m	32mb
cpm-jvm-heap	elasticsearch.node.stats	jvm.mem.heap.used.pct per cluster	15m	16mb
cpm-shard-count	elasticsearch.cluster.stats	shards.count per cluster	15m	8mb
cpm-cluster-event-rate	elasticsearch.node.stats	thread_pool.write.queue.count per cluster	15m	16mb

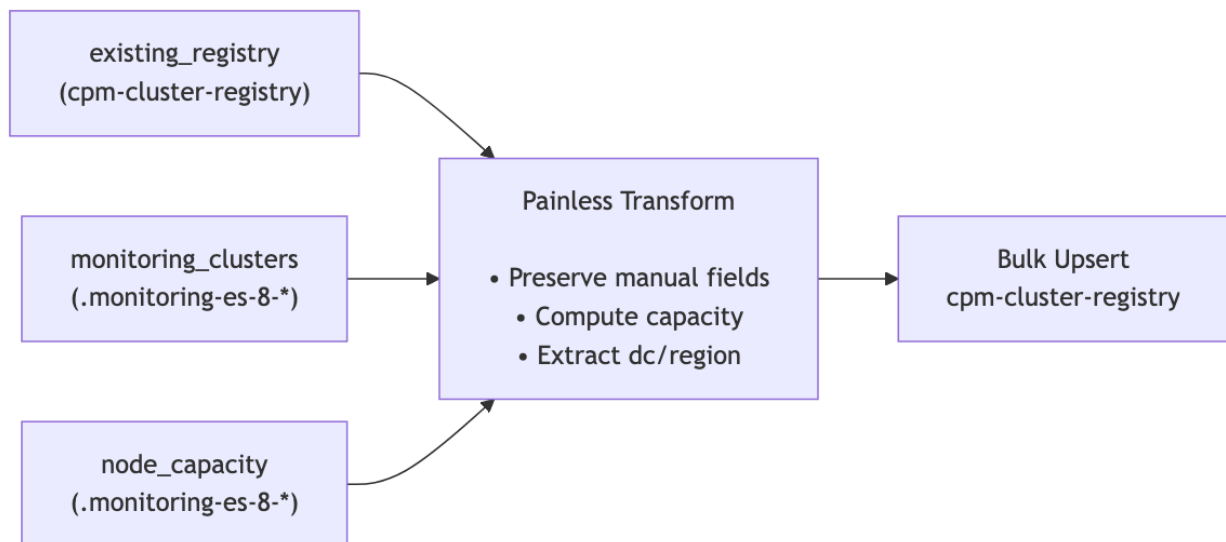
Each job has a corresponding datafeed reading from `.monitoring-es-8-*` with aggregations over 15-minute buckets. All jobs use `model_prune_window: 30d` and `summary_count_field_name: doc_count`.

The `cpm-forecast-trigger` watcher kicks off 24-hour forecasts on four of these jobs every hour.

Watchers

1. cpm-registry-sync

Schedule: Daily at 00:00 UTC **Purpose:** Synchronizes the cluster registry with live monitoring data.



Key logic: - Merges existing registry entries with live monitoring data - Preserves manually-set fields: `cluster_id`, `ingest_hosts`, `ingest_capacity_per_day`, `active` - Computes capacity: `disk_total_bytes` (85% of raw), `heap_max_bytes`, `shard_max_threshold` (20 shards/GB heap), `write_queue_threshold` (200) - Extracts `dc` from node attributes (`dc` or `region`), falling back to existing value

2. cpm-forecast-trigger

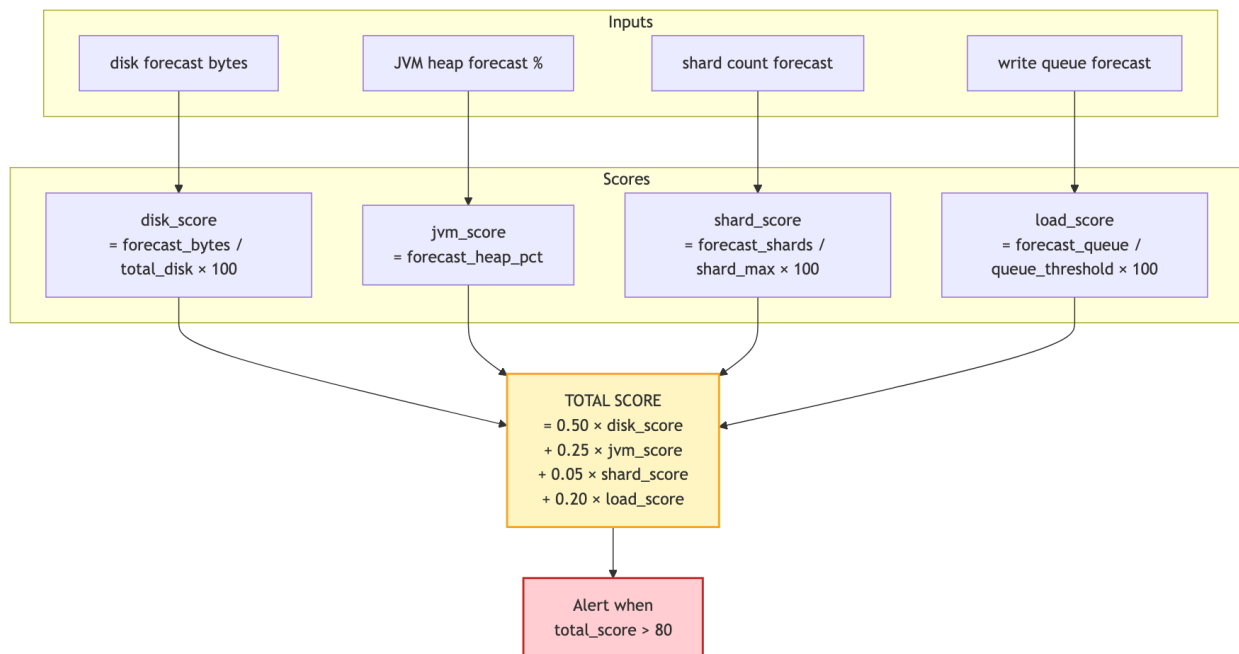
Schedule: Every 1 hour **Purpose:** Triggers 24-hour ML forecasts on four anomaly detection jobs.

Posts `{"duration":"24h","expires_in":"48h"}` to: - `/_ml/anomaly_detectors/cpm-store-size/_forecast` - `/_ml/anomaly_detectors/cpm-jvm-heap/_forecast` - `/_ml/anomaly_detectors/cpm-shard-count/_forecast` - `/_ml/anomaly_detectors/cpm-cluster-event-rate/_forecast`

3. cpm-scoring

Schedule: 00:05 UTC daily **Purpose:** Computes a weighted composite health score per cluster from ML forecasts.

Scoring formula:



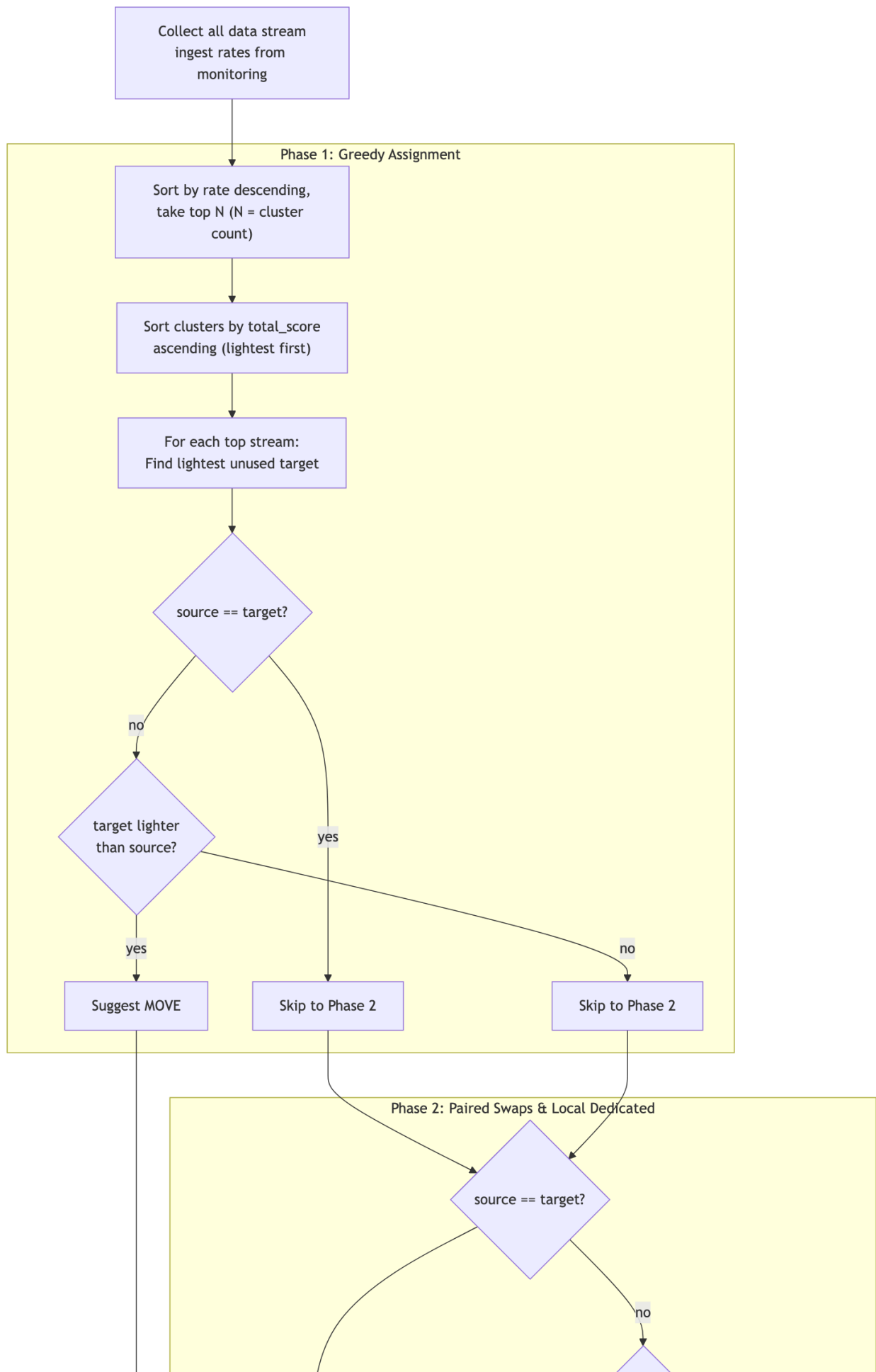
Inputs: - registry — cluster capacity thresholds from cpm-cluster-registry - disk_forecast — peak forecasted disk usage per cluster (from .ml-anomalies-*) - jvm_forecast — peak forecasted JVM heap % per cluster - shard_forecast — peak forecasted shard count per cluster - load_forecast — peak forecasted write queue depth per cluster

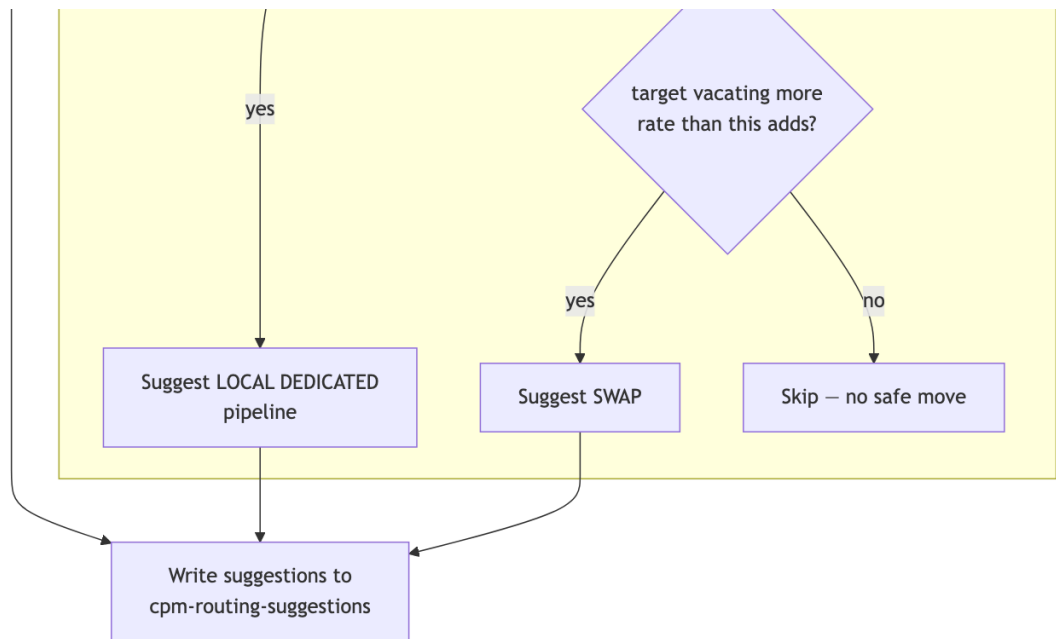
Output: Writes scored document to cpm-scores

4. cpm-routing-advisor

Schedule: 00:10 UTC daily **Purpose:** Recommends which high-volume datasets should get dedicated pipelines or be moved to less loaded clusters.

Algorithm (two-phase):



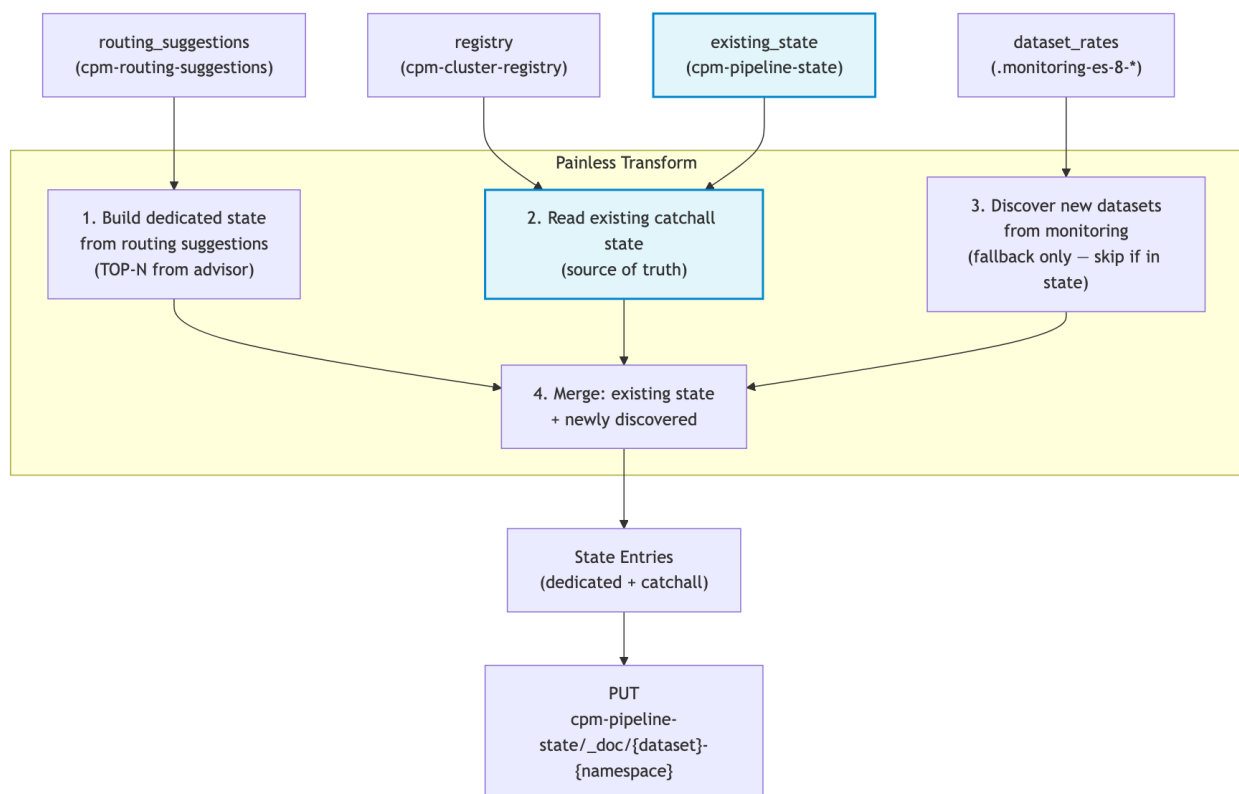


Inputs: - `scores` — latest cluster health scores from `cpm-scores` - `index_rates` — per-cluster per-index ingest rates from `.monitoring-es-8-*`

Output: Bulk writes suggestions to `cpm-routing-suggestions`

5. cpm-state-manager

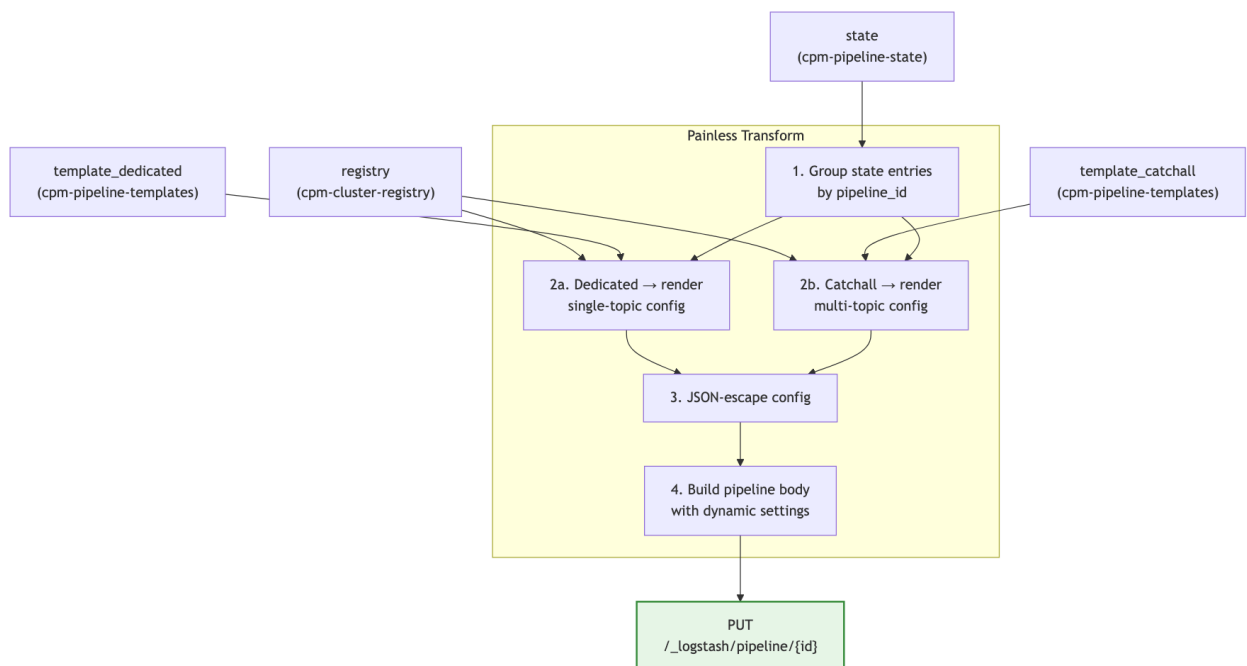
Schedule: 00:15 UTC daily **Purpose:** Computes desired pipeline state — which dataset goes to which cluster/pipeline type. **State is the source of truth for catchall assignments.**



Key principles: - **Dedicated pipelines always win** — routing suggestions override state - **State is authoritative for catchall** — if state says dataset X → cluster A , that assignment persists - **Monitoring is fallback only** — datasets not in state are discovered from monitoring and assigned to their current cluster - **New datasets auto-join** — appear in the appropriate cluster's catchall and get written to state

6. cpm-pipeline-manager

Schedule: Manual trigger only (cron `0 59 23 31 2 ?` — Feb 31 never occurs) **Purpose:** Reads desired state, renders Logstash pipeline configs from templates, and pushes to the Logstash management API.



Template placeholder substitution:

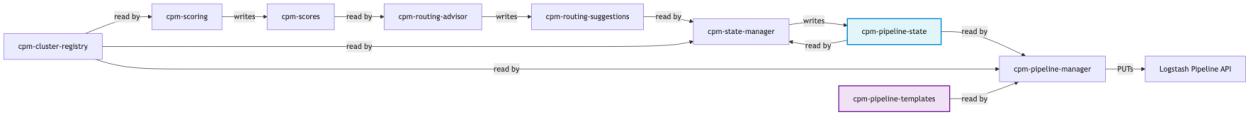
Placeholder	Dedicated	Catchall	Source
<code>__KAFKA_BOOTSTRAP__</code>	Kafka bootstrap servers	Kafka bootstrap servers	Template doc <code>kafka_bootstrap</code> field
<code>__TOPIC__</code>	Single topic (dataset-namespace)	—	State entry
<code>__TOPICS_LIST__</code>	—	Comma-separated quoted topic list	State entries
<code>__PIPELINE_ID__</code>	Pipeline ID (e.g. dc_cpm-dedicated-clusterId)	—	Derived from state
<code>__CLUSTER_ID__</code>	—	Cluster ID	State entry
<code>__GROUP_ID__</code>	Kafka group ID (cpm-{clusterName})	Kafka group ID (cpm-{clusterName})	Registry <code>cluster_name</code>
<code>__CONSUMER_THREADS__</code>	Kafka consumer thread count	Kafka consumer thread count	Template doc <code>consumer_threads</code> field
<code>__ES_HOSTS__</code>	Formatted ES hosts list	Formatted ES hosts list	Registry <code>ingest_hosts</code>
<code>__DATASET__</code>	Data stream dataset	—	State entry
<code>__NAMESPACE__</code>	Data stream namespace	—	State entry
<code>__API_KEY_VAR__</code>	Keystore variable name	Keystore variable name	Derived from <code>cluster_id</code>

Pipeline settings (from template doc, dynamic): - `pipeline.workers` - `pipeline.batch.size` - `queue.type` - `queue.max_bytes`

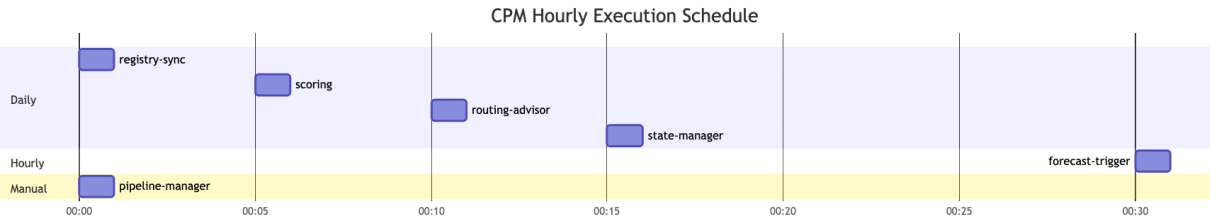
Indices

Index	Purpose	Key Fields
cpm-cluster-registry	Cluster topology & capacity	cluster_uuid , cluster_id , cluster_name , active , dc , ingest_hosts , disk_total_bytes , heap_max_bytes , node_count , shard_max_threshold , write_queue_threshold
cpm-routing-config	Manual routing overrides	cluster_id , dataset , locked , previous_cluster_id
cpm-scores	Composite health scores	scored_at , forecast_horizon_hours , clusters[] (nested: cluster_id , total_score , disk_score , jvm_score , shard_score , load_score , alert)
cpm-bytes-per-event	Average bytes per event	bytes_per_event , computed_at , total_disk_delta_bytes , total_events
cpm-routing-suggestions	Routing recommendations	dataset , namespace , source_cluster_id , suggested_cluster_id , source_score , target_score , event_rate_1h , reason
cpm-pipeline-templates	Logstash config templates	name , template , kafka_bootstrap , consumer_threads , pipeline_settings
cpm-pipeline-state	Desired pipeline assignments	dataset , namespace , pipeline_type , pipeline_id , cluster_id , dc , topic , updated_at

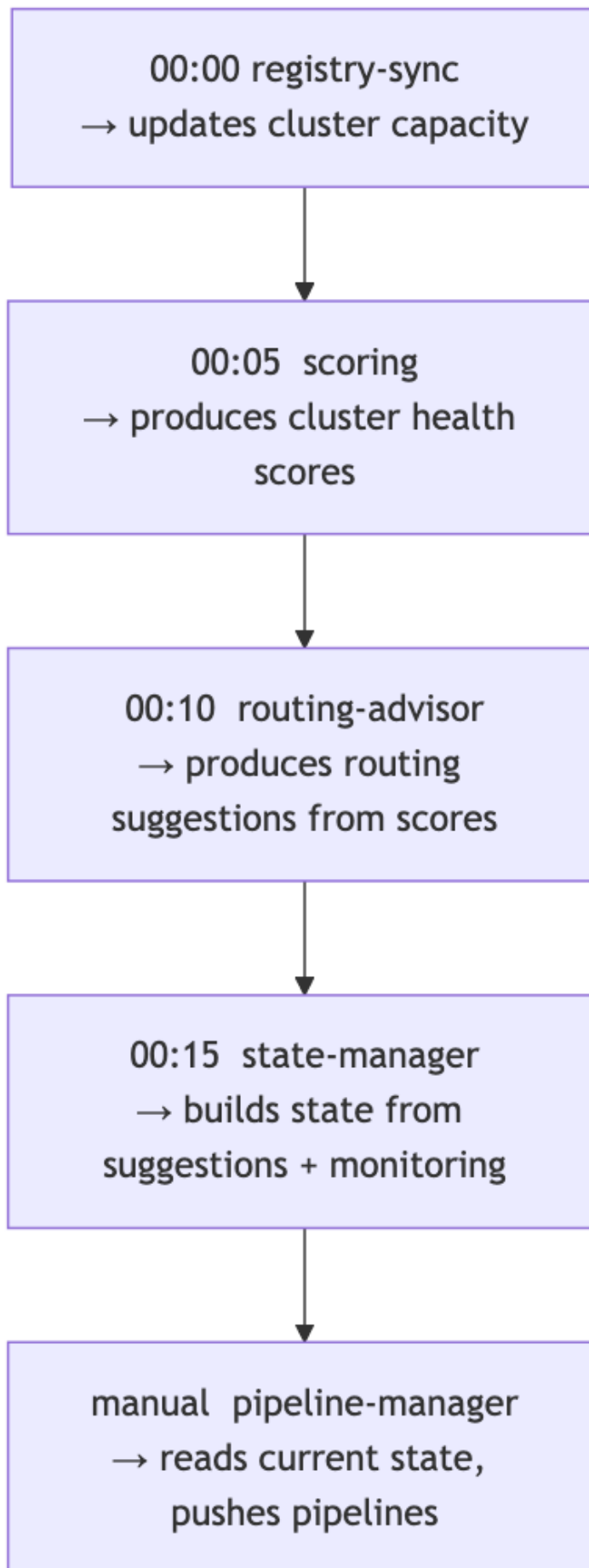
Index relationship diagram:



Execution Schedule



Timing dependency chain:



Note: All daily watchers run in sequence: registry-sync (00:00) → scoring (00:05) → routing-advisor (00:10) → state-manager (00:15). Only `cpm-forecast-trigger` runs hourly; the pipeline-manager is manual-only.

Pipeline State Model

`cpm-pipeline-state` is the source of truth for catchall topic assignments.

State Entry Structure

```
{
  "dataset": "endpoint.events.process",
  "namespace": "default",
  "pipeline_type": "dedicated",
  "pipeline_id": "unknown-region_cpm-dedicated-A9F0WH3jRSycLLvcAF6dtg",
  "cluster_id": "A9F0WH3jRSycLLvcAF6dtg",
  "dc": "unknown-region",
  "topic": "endpoint.events.process-default",
  "updated_at": "2026-05-18T10:05:00.000Z"
}
```

Assignment Rules

1. DEDICATED (highest priority)

TOP-N datasets by ingest
rate

Always override existing
state

One dedicated pipeline
per cluster

2. EXISTING STATE (source of truth for catchall)

If dataset in state → keep
its assignment

User edits to cluster_id
are preserved

Allows manual dataset-to-cluster moves

3. MONITORING DATA (fallback for new datasets)

Datasets not in state
discovered from
.monitoring-es-8-* index
stats

Assigned to their current
cluster

Written to state on next
state-manager run

To move a dataset from cluster A's catchall to cluster B's:

1. Edit the state document: `PUT cpm-pipeline-state/_doc/endpoint.events.process-default {
 "cluster_id": "cluster-B-id", ... }`

2. Run the state-manager (or wait for hourly run): `POST _watcher/watch/cpm-state-manager/_execute`
 3. Verify the state entry has the new `cluster_id`
 4. Push pipelines: `POST _watcher/watch/cpm-pipeline-manager/_execute`
 5. Verify cluster B's catchall pipeline now includes the moved topic
-

Template System

Pipeline configurations are rendered from templates stored in `cpm-pipeline-templates`.

Dedicated Template (`_id: dedicated`)

```
input {
  kafka {
    bootstrap_servers => "__KAFKA_BOOTSTRAP__"
    topics => ["__TOPIC__"]
    group_id => "__GROUP_ID__"
    consumer_threads => __CONSUMER_THREADS__
    codec => json
    decorate_events => true
  }
}

output {
  elasticsearch {
    hosts => [__ES_HOSTS__]
    data_stream => true
    data_stream_type => "logs"
    data_stream_dataset => "__DATASET__"
    data_stream_namespace => "__NAMESPACE__"
    api_key => "${__API_KEY_VAR__}"
    ssl_certificate_verification => true
  }
}
```

Catchall Template (`_id: catchall`)

```
input {
  kafka {
    bootstrap_servers => "__KAFKA_BOOTSTRAP__"
    topics => [__TOPICS_LIST__]
    group_id => "__GROUP_ID__"
    consumer_threads => __CONSUMER_THREADS__
    codec => json
    decorate_events => true
  }
}

output {
  elasticsearch {
    hosts => [__ES_HOSTS__]
    data_stream => true
    api_key => "${__API_KEY_VAR__}"
    ssl_certificate_verification => true
  }
}
```

Template Document Structure

```
{
  "name": "dedicated",
  "description": "Template for per-dataset Logstash pipelines managed by CPM",
  "kafka_bootstrap": "kafka:9092",
  "consumer_threads": 1,
  "pipeline_settings": {
    "pipeline.workers": 4,
    "pipeline.batch.size": 125,
    "queue.type": "persisted",
    "queue.max_bytes": "1gb"
  },
  "template": "<logstash config with __PLACEHOLDERS__>"
}
```

Configurable per-template fields: - `kafka_bootstrap` — Kafka bootstrap servers (default: `kafka:9092`) - `consumer_threads` — Kafka consumer thread count (default: `1`) - `pipeline_settings` — Logstash pipeline settings object (any valid settings)

Changes to these fields take effect on the next pipeline-manager execution.

Configuration & Deployment

Master Configuration File

Deployment

```
# Deploy a single watcher
curl -X PUT "https://<host>/_watcher/watch/<name>" \
  -H "Authorization: ApiKey <key>" \
  -H "Content-Type: application/json" \
  -d @watcher.json

# Execute a watcher manually
curl -X POST "https://<host>/_watcher/watch/<name>/_execute" \
  -H "Authorization: ApiKey <key>"

# Deploy ML job
curl -X PUT "https://<host>/_ml/anomaly_detectors/<job>" \
  -H "Authorization: ApiKey <key>" \
  -d @job.json

# Start datafeed
curl -X POST "https://<host>/_ml/datafeeds/<feed>/_start" \
  -H "Authorization: ApiKey <key>"
```

Operational Guide

Manual Pipeline Push

```
# 1. Build state
curl -X POST "https://<host>/_watcher/watch/cpm-state-manager/_execute" \
  -H "Authorization: ApiKey <key>"

# 2. Review state (optional)
curl "https://<host>/cpm-pipeline-state/_search?size=500" \
  -H "Authorization: ApiKey <key>"

# 3. Edit state if needed (move datasets between clusters)
curl -X PUT "https://<host>/cpm-pipeline-state/_doc/<dataset>--<namespace>" \
  -H "Authorization: ApiKey <key>" \
  -H "Content-Type: application/json" \
  -d '{ ... updated state ... }'

# 4. Push pipelines
curl -X POST "https://<host>/_watcher/watch/cpm-pipeline-manager/_execute" \
  -H "Authorization: ApiKey <key>"

# 5. Verify
curl "https://<host>/_logstash/pipeline/" \
  -H "Authorization: ApiKey <key>"
```

Changing Pipeline Settings

Edit the template document in ES — no code changes needed:

```
# Increase consumer threads
curl -X PUT "https://<host>/cpm-pipeline-templates/_doc/dedicated" \
-H "Authorization: ApiKey <key>" \
-H "Content-Type: application/json" \
-d '{
  ...
  "consumer_threads": 4,
  "pipeline_settings": {
    "pipeline.workers": 8,
    "pipeline.batch.size": 250,
    "queue.type": "persisted",
    "queue.max_bytes": "2gb"
  },
  ...
}'

# Then re-push pipelines
curl -X POST "https://<host>/_watcher/watch/cpm-pipeline-manager/_execute" \
-H "Authorization: ApiKey <key>"
```

Adding a New Cluster

1. Ensure the cluster appears in `.monitoring-es-8-*` monitoring data
2. Run registry-sync: `POST _watcher/watch/cpm-registry-sync/_execute`
3. Verify in `cpm-cluster-registry` — set `ingest_hosts` and `active: true` if needed
4. Run state-manager to assign datasets
5. Push pipelines

Monitoring Health

```
# Check cluster scores
GET cpm-scores/_search?sort=scored_at:desc&size=1

# Check which clusters have alerts
GET cpm-scores/_search
{ "query": { "nested": { "path": "clusters", "query": { "term": { "clusters.alert": true } } } }

# Check current routing suggestions
GET cpm-routing-suggestions/_search?sort=suggested_at:desc&size=10

# Check pipeline state
GET cpm-pipeline-state/_search?size=500

# Verify deployed pipelines
GET _logstash/pipeline/
```

Painless Development Notes

ES Watcher Painless scripts have several constraints that differ from standard Painless:

Constraint	Workaround
No <code>String.split(String)</code>	Use <code>.substring()</code> and <code>.lastIndexOf()</code> manually
No <code>String.replaceAll(regex, repl)</code>	Use <code>.replace()</code> for literal replacements only
No user-defined functions	Inline all logic
Special characters in strings	Use <code>Character.toString((char)N)</code> (newline=10, tab=9, etc.)
<code>Math.min()</code> class cast with mixed types	Use ternary operator: <code>a < b ? a : b</code>
<code>::es_redacted::</code> in re-PUT watchers	Use explicit <code>ApiKey</code> auth in webhook headers; use search inputs instead of HTTP inputs for reading ES data
<code>::es_host::</code> in <code>_execute</code>	Use explicit hostname; <code>::es_host::</code> only resolves during scheduled execution
Watcher redacts <code>Authorization</code> headers	ES automatically replaces <code>Authorization</code> header values with <code>::es_redacted::</code> on storage — use search inputs instead of HTTP inputs for reading from ES indices

JSON Escaping in Painless

Pipeline configs must be JSON-escaped before embedding in the Logstash PUT body:

```
String esc = config
    .replace(BS, BS + BS)      // \ → \\
    .replace(DQ, BS + DQ)     // " → \"
    .replace(NL, BS + 'n')    // newline → \n
    .replace(CR, BS + 'r')    // carriage return → \r
    .replace(TAB, BS + 't');  // tab → \t
```

Pipeline Settings Serialization

Numeric values must be unquoted, strings must be quoted:

```
if (sval instanceof Number) {
    settingsStr = settingsStr + DQ + skey + DQ + ':' + String.valueOf(sval);
} else {
    settingsStr = settingsStr + DQ + skey + DQ + ':' + DQ + String.valueOf(sval) + DQ;
}
```

Mustache Template Pitfalls

Closing `}` must come **after** all conditional sections:

```
// WRONG – closes JSON object before conditional fields
"active":{{active}}{{#ingest_hosts}}"ingest_hosts":"{{ingest_hosts}}",{{/ingest_hosts}}

// RIGHT – closing } after all conditionals
"active":{{active}}{{#ingest_hosts}}, "ingest_hosts":"{{ingest_hosts}}">{{/ingest_hosts}}
```